



Nội dung

1. Quy trình viết mã nguồn
2. Tái cấu trúc mã nguồn
3. Coding conventions
4. Quản lý mã nguồn

2

Nội dung

1. Quy trình viết mã nguồn
2. Tái cấu trúc mã nguồn
3. Coding conventions
4. Quản lý mã nguồn

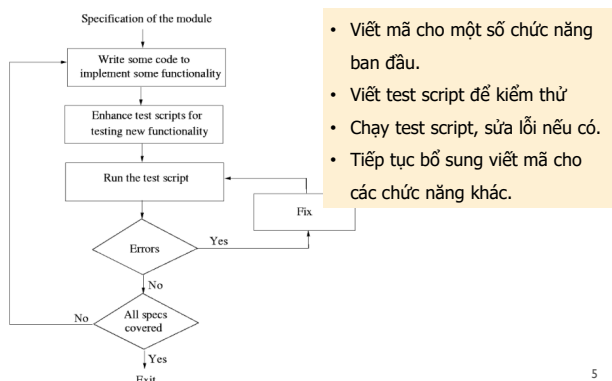
3

Giới thiệu

- Hoạt động lập trình được bắt đầu khi các thiết kế, các đặc tả về module cần xây dựng đã được thực hiện.
- Một số quy trình viết mã được các lập trình viên áp dụng:
 - Quy trình viết mã tăng dần (Incremental Coding Process)
 - Quy trình viết mã hướng kiểm thử (Test-Driven Development)

4

Quy trình viết mã tăng dần Incremental Coding Process



5

Quy trình viết mã tăng dần Incremental Coding Process

• Điểm mạnh:

- **Dễ debug (dễ phát hiện lỗi):** vì lập trình viên viết và chạy test script sau mỗi lần viết mã cho chức năng mới, nên nếu chạy test script phát hiện lỗi, thì lỗi này thuộc về phần mã mới bổ sung vào gần nhất.
- Việc viết test script có ý nghĩa rất lớn khi mã lệnh càng ngày càng nhiều: **test script giúp kiểm tra nhanh, chính xác và thường xuyên.**
- Test script là cơ sở để thực hiện **kiểm thử đơn vị (unit testing)** trước khi đưa module lên server (repository)

6

Quy trình viết mã tăng dần Incremental Coding Process

• Hạn chế:

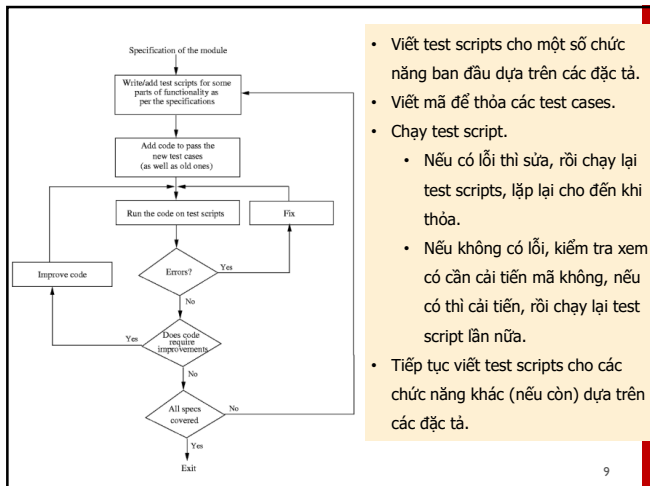
- Giữa **mã nguồn** và **test scripts** thường **không đồng bộ**:
Viết mã rất nhiều, nhưng test scripts lại ít, **không đủ độ phủ** kiểm tra hết các đoạn mã đã viết ra.

7

Quy trình viết mã hướng kiểm thử Test-Driven Development - TDD

- TDD là quy trình viết mã ngược với hướng tiếp cận thông thường khi viết mã:
 - Đầu tiên, viết các test cases để kiểm tra mã trước.
 - Sau đó mới viết mã để thỏa các test cases này.

8



9

Quy trình viết mã hướng kiểm thử Test-Driven Development - TDD

• Điểm mạnh:

- Mã nguồn **luôn đồng bộ** với test scripts.
- Vì phải viết test cases dựa trên các đặc tả, và viết code sau đó để thỏa các test cases nên hệ thống xây dựng ra sẽ **đáp ứng tốt các yêu cầu đặt ra**.
- Những **chức năng quan trọng** thường được viết test script trước, nên chúng sẽ được lập trình trước và được **kiểm thử nhiều nhất**.

10

Quy trình viết mã hướng kiểm thử Test-Driven Development - TDD

• Hạn chế:

- **Tính đầy đủ** của mã nguồn **phụ thuộc** vào tính toàn diện, đầy đủ của bộ test cases được thiết kế. Thông thường, khó có thể viết các test cases một cách đầy đủ (thường sót đối với các trường hợp đặc biệt).
- Đôi khi việc viết mã để thỏa test case sau lại phải dẫn đến việc **thay đổi thuật toán, các đoạn mã đã có**.

11

Nội dung

1. Quy trình viết mã nguồn
2. Tái cấu trúc mã nguồn
3. Coding conventions
4. Quản lý mã nguồn

12

Tái cấu trúc mã nguồn (Cải tiến mã nguồn) **Refactoring**

- Trong quá trình phát triển hệ thống, mã nguồn luôn thường xuyên bị thay đổi:
 - Do thêm tính năng mới
 - Do những thay đổi yêu cầu
- Cho dù đã được thiết kế tốt, thì sau một thời gian với những thay đổi, bổ sung vào mã nguồn, cấu trúc của chúng dần trở nên **phức tạp** và **không tốt**. Dẫn đến:
 - Đọc mã nguồn khó hơn (cấu trúc phức tạp, lộn xộn)
 - Khó tìm lỗi hơn (khi xảy ra vấn đề)
 - Khó khăn cho việc đáp ứng các thay đổi yêu cầu.
 - Về tổng thể: chất lượng, hiệu suất giảm, chi phí sản xuất tăng.

13

Tái cấu trúc mã nguồn

- Refactoring** là một kỹ thuật cho việc tái cấu trúc mã nguồn đã có, thay đổi cấu trúc bên trong mà **không làm thay đổi hành vi bên ngoài**.
- Theo định nghĩa của Martin Fowler** (<https://refactoring.com/>): Refactoring là việc làm thay đổi cấu trúc bên trong của phần mềm, làm cho nó dễ hiểu, chi phí sửa đổi thấp mà không làm thay đổi hành vi bên ngoài.

14

Tái cấu trúc mã nguồn

- Mục tiêu cơ bản của Refactoring** là **cải tiến thiết kế** (ở giai đoạn viết mã, không phải ở giai đoạn thiết kế).
 - Kết quả: làm tăng cohesion và giảm coupling, tuân theo nguyên lý đóng-mở tốt hơn.
- Đối tượng của Refactoring** là: mã nguồn.
- Rủi ro khi làm refactoring** là: làm hỏng hệ thống.
Để giảm thiểu rủi ro, cần tuân theo 2 nguyên tắc:
 - Tái cấu trúc từng bước nhỏ một.
 - Có test scripts để kiểm tra lại sau khi thay đổi.

15

Tái cấu trúc mã nguồn

- Khi nào thì cần thực hiện Refactoring?** Đó là khi trong mã nguồn có một số dấu hiệu không tốt (gọi là “bad smells”):
 - Trùng mã (**Duplicate code**)
 - Phương thức quá dài (**Long method**)
 - Tham số phương thức quá nhiều (**Long parameter list**)
 - Lớp quá dài (**Long class**)
 - ...

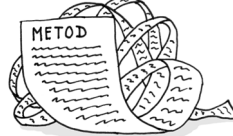
16

Tái cấu trúc mã nguồn

- Một số ví dụ về Refactoring**
 - Nguồn: <https://sourcemaking.com/refactoring/smells>

17

Long methods



Phương thức quá dài

- Khó đọc, khó bảo trì
- Nếu > 10 dòng code → nên đặt câu hỏi
- Nếu trong phương thức cần có comment → nên suy nghĩ tách phương thức
- Giải pháp:**

- Extract method**

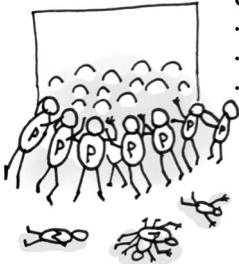
```
void printOwing() {  
    // Print banner  
    System.out.println("Title");  
    System.out.println("Sub-title");  
  
    // Print details  
    System.out.println("name: " + name);  
    System.out.println("amount: " +  
        getOutstanding());  
}
```



```
void printOwing() {  
    printBanner();  
  
    printDetails();  
}
```

18

METHOD



Long parameter list

Quá nhiều tham số

- Khó đọc, khó bảo trì
- Mỗi lần gọi, phải quan tâm thứ tự tham số
- Giải pháp:
 - Introduce Parameter Object
 - Replace Parameter with Method Call
 - Preserve Whole Object

19

Introduce Parameter Object

```
public void themSV(String maSV, String hoTen, bool gioiTinh, DateTime
ngaySinh, String noiSinh, String sdt, int maKhoa...) {
    ...
}
```

```
public class SinhVien {
    String maSV, hoTen, noiSinh, sdt;
    bool gioiTinh;
    DateTime ngaySinh;
    ...
}
```

```
SinhVien sv = new SinhVien();
sv.setMaSV("012345");
sv.setHoTen("Nguyen A");
...
```

themSV(sv);

20

Introduce Parameter Object

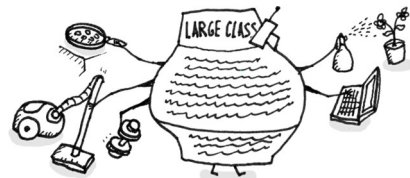
Customer
amountInvoicedIn (start : Date, end : Date)
amountReceivedIn (start : Date, end : Date)
amountOverduelIn (start : Date, end : Date)

Your methods contain a repeating group of parameters

Customer
amountInvoicedIn (date : DateRange)
amountReceivedIn (date : DateRange)
amountOverduelIn (date : DateRange)

Replace these parameters with an object.

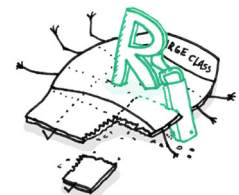
21



Large class

Lớp quá lớn, chứa nhiều fields, methods, lines of code

- Khó bảo trì
- Giải pháp:
 - Chia nhỏ class

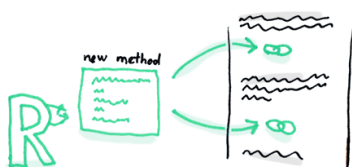
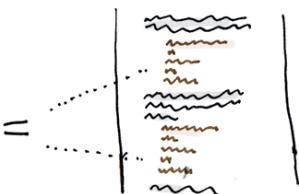


22

Duplicate code

Mã nguồn lặp lại

- Khó bảo trì
- Giải pháp:
 - Extract method



23

Ví dụ: Hãy refactoring đoạn mã sau

```
int main()
{
    int n;
    int a[30];

    // Nhập mảng
    do{
        cout<<"Nhập số phần tử: ";
        cin>>n;
        if(n<0||n>30)
            cout<<"Nhập lại số phần tử.\n";
        else break;
    } while(true);
    for(int i=0;i<n;i++)
    {
        cout<<"Phần tử thu "<<i<<": ";
        cin>>a[i];
    }
```

```
// Hiển thị mảng
for(int i=0;i<n;i++)
    cout<<a[i]<<" ";
```

```
// Sắp xếp mảng tăng dần
int temp;
for(int i=0;i<n-1;i++)
    for(int j=i+1;j<n;j++)
    {
        if(a[i]>a[j])
        {
            a[i]=temp;
            a[i]=a[j];
            a[j]=temp;
        }
    }
```

```
// In mảng kết quả
cout<<"\n Mảng kết quả:"<<endl;
for(int i=0;i<n;i++)
    cout<<a[i]<<" ";
getch();
}
```

24

```
int main()
{
    int n;
    int a[30];
    nhapmang(a,n);
    hienmang(a,n);
    sapxetangdan(a,n);
    cout<<"\n Mang ket qua:"<<endl;
    hienmang(a,n);
    getch();
}
```

```
void nhapmang(int a[],int &n)
{
}

void sapxetangdan(int a[],int n)
{
}

void hienmang(int a[],int n)
{
}
```

25

Nội dung

1. Quy trình viết mã nguồn
2. Tái cấu trúc mã nguồn
3. Coding conventions
4. Quản lý mã nguồn

26

Coding conventions

```
public void Test()
{
    for (int i = 1; i < 101; i++)
    {
        if (i % 3 == 0 && i % 5 == 0)
        {
            Console.WriteLine("FizzBuzz");
        }
        else if (i % 3 == 0)
        {
            Console.WriteLine("Fizz");
        }
        else if (i % 5 == 0)
        {
            Console.WriteLine("Buzz");
        }
        else
        {
            Console.WriteLine(i);
        }
    }
}
```

What do you think?
Do we need to make
the code better?

Nguồn:
<https://www.codeproject.com/Articles/539179/Some-practices-to-write-better-Csharp-NET-code>

27

Coding conventions

```
public void Check()
{
    for (int i = 1; i <= 100; i++)
    {
        string output = "";
        if (i % 3 == 0) { output = "Fizz"; }
        if (i % 5 == 0) { output = output + "Buzz"; }
        if (output == "") { output = i.ToString(); }
        Console.WriteLine(output);
    }
}
```

What do you think now ?
Do we need to make the code even better?

28

```
public void DoFizzBuzz()
{
    for (int number = 1; number <= 100; number++)
    {
        var output = GetFizzBuzzOutput(number);
        Console.WriteLine(output);
    }
}

private static string GetFizzBuzzOutput(int number)
{
    string output = string.Empty;
    if (number%3 == 0)
    {
        output = "Fizz";
    }
    if (number%5 == 0)
    {
        output += "Buzz";
    }
    if (string.IsNullOrEmpty(output))
    {
        output = number.ToString();
    }
    return output;
}
```

29

Coding conventions

- Anybody can write code. With a few months of programming experience, you can write 'working applications'. **Making it work is easy**, but **doing it the right way requires more work**, than just making it work.
 - Majority of the programmers write '**working code**', but not '**good code**'.
- Writing 'good code' is an **art** and you must learn and practice it.

Nguồn: <https://www.dotnetspider.com/tutorials/BestPractices.aspx>

30

Coding conventions

- The characteristics of 'good code':
 - Reliable
 - Maintainable
 - Efficient
- Most of the developers are inclined towards writing code for higher performance, compromising reliability and maintainability.
- But considering the long term ROI (Return On Investment), efficiency and performance comes below reliability and maintainability.
 - If your code is not reliable and maintainable, you (and your company) will be spending lot of time to identify issues, trying to understand code etc throughout the life of your application.

31

Coding conventions

- Programmers spend far more time reading code than writing code. Over the life of the code, the author spends a considerable time reading it during debugging and enhancement.
- People other than the author also spend considerable effort in reading code because the code is often maintained by someone other than the author.

32

Coding conventions

- In short, it is of prime importance to write code in a manner that is easy to read and understand.
- Coding conventions provide rules and guidelines for some aspects of programming in order to make code easier to read.
- Most organizations who develop software regularly develop their own standards.

33

Coding conventions

- In general, coding standards provide guidelines for programmers regarding:
 - naming,
 - file organization,
 - statements and declarations,
 - and layout and comments.
- Coding standards, called:
 - Coding conventions
 - Coding style guidelines

34

Coding conventions

- Software programmers are highly recommended to follow these guidelines to help improve the readability of their source code and make software maintenance easier.

Nguồn: https://en.wikipedia.org/wiki/Coding_conventions

35

Coding conventions

- Some references
 - <https://www.dotnetspider.com/tutorials/BestPractices.aspx>
 - <https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/inside-a-program/coding-conventions>
 - <https://www.dofactory.com/reference/csharp-coding-standards>
 - <https://www.codeproject.com/Articles/539179/Some-practices-to-write-better-Csharp-NET-code>
 - <https://docs.microsoft.com/en-us/dotnet/standard/design-guidelines/index>
 - <https://csharpcodingguidelines.com>

36

Naming Conventions - C#

use **PascalCase** for **class** names and **method** names.

use **camelCase** for **method arguments** and **local variables**.

```
public class UserLog
{
    public void Add(LogEvent logEvent)
    {
        int itemCount = logEvent.Items.Count;
        // ...
    }
}
```

Why: consistent with the Microsoft's .NET Framework and easy to read.

37

Naming Conventions - C#

- use **noun** or **noun phrases** to name a **class**.

```
public class Employee
{
}
public class BusinessLocation
{
}
public class DocumentCollection
{
}
```

AVOID

```
public class AddEmployee
{
}
```

Why: consistent with the Microsoft's .NET Framework and easy to remember.

38

Naming Conventions - C#

- **DO NOT** use **Hungarian** notation or any other type identification in identifiers

```
// Correct
int counter;
string name;

// Avoid
int iCounter;
string strName;
```

Why: consistent with the Microsoft's .NET Framework and Visual Studio IDE makes determining types very easy (via tooltips). In general you want to avoid type indicators in any identifier

39

Naming Conventions - C#

- **DO NOT** use **Screaming Caps** for constants or read-only variables

```
// Correct
public static const string ShippingType = "DropShip";

// Avoid
public static const string SHIPPINGTYPE = "DropShip";
```

Why: consistent with the Microsoft's .NET Framework. Caps grab too much attention.

40

Naming Conventions - C#

- **AVOID** using **Abbreviations**.
- Exceptions: abbreviations commonly used as names, such as **Id**, **Xml**, **Ftp**, **Uri**

```
// Correct
UserGroup userGroup;
Assignment employeeAssignment;

// Avoid
UserGroup usrGrp;
Assignment empAssignment;

// Exceptions
CustomerId customerId;
XmlDocument xmlDocument;
FtpHelper ftpHelper;
UriPart uriPart;
```

Why: consistent with the Microsoft's .NET Framework and prevents inconsistent abbreviations.

41

Naming Conventions - C#

- use **PascalCasing** for **abbreviations** 3 characters or more (2 chars are both uppercase)

```
HtmlHelper htmlHelper;
FtpTransfer ftpTransfer;
UiControl uiControl;
```

```
HTMLHelper
FTPTransfer
UiControl
```



good



not good

Why: consistent with the Microsoft's .NET Framework. Caps would grab visually too much attention.

42

Naming Conventions - C#

- **DO NOT** use **Underscores** in identifiers.
Exception: you can prefix private static variables with an underscore.

```
// Correct
public DateTime clientAppointment;
public TimeSpan timeLeft;

// Avoid
public DateTime client_Appointment;
public TimeSpan time_Left;

// Exception
private DateTime _registrationDate;
```

Why: consistent with the Microsoft's .NET Framework and makes code more natural to read (without 'slur'). Also avoids underline stress (inability to see underline).

43

Naming Conventions - C#

- use **predefined type names** instead of **system type names** like `Int16`, `Single`, `UInt64`, etc

```
// Correct
string firstName;
int lastIndex;
bool isSaved;

// Avoid
String firstName;
Int32 lastIndex;
Boolean isSaved;
```

Why: consistent with the Microsoft's .NET Framework and makes code more natural to read.

44

Naming Conventions - C#

- prefix interfaces with the **letter I**. Interface names are **noun (phrases)** or **adjectives**.

```
public interface IShape
{
}
public interface IShapeCollection
{
}
public interface IGroupable
{
}
```

Why: consistent with the Microsoft's .NET Framework.

45

Naming Conventions - C#

Capitalize SQL Special Words

to distinguish them from your table and column names

```
01 SELECT id, username FROM user;
02
03 UPDATE user SET last_login = NOW()
04 WHERE id = '123'
05
06 SELECT id, username FROM user u
07 LEFT JOIN user_address ua ON(u.id = ua.user_id)
08 WHERE ua.state = 'NY'
09 GROUP BY u.id
10 ORDER BY u.username
11 LIMIT 0,20
```

46

Layout Conventions - C#

- Good layout uses formatting to emphasize the structure of your code and to make the code easier to read.
- Microsoft examples and samples conform to the following conventions:
 - Use the default Code Editor settings (smart indenting, four-character indents, tabs saved as spaces). For more information, see Options, Text Editor, C#, Formatting.
 - Write only one statement per line.
 - Write only one declaration per line.

47

Layout Conventions - C#

- Layout conventions (cont)
 - Add at least one blank line between method definitions and property definitions.
 - Use parentheses to make clauses in an expression apparent.

```
C#
if ((val1 > val2) && (val1 > val3))
{
    // Take appropriate action.
}
```

48

Layout Conventions

Consistent Indentation

Style 1:

```
1 function foo() {
2   if ($maybe) {
3     do_it_now();
4     again();
5   } else {
6     abort_mission();
7   }
8   finalize();
9 }
```

Style 2:

```
01 function foo()
02 {
03     if ($maybe)
04     {
05         do_it_now();
06         again();
07     }
08     else
09     {
10         abort_mission();
11     }
12     finalize();
13 }
```

More styles: https://en.wikipedia.org/wiki/Indent_style

49

Layout Conventions - C#

- Declare all member variables at the top of a class, with static variables at the very top.

```
// Correct
public class Account
{
    public static string BankName;
    public static decimal Reserves;

    public string Number {get; set;}
    public DateTime DateOpened {get; set;}
    public DateTime DateClosed {get; set;}
    public decimal Balance {get; set;}

    // Constructor
    public Account()
    {
        // ...
    }
}
```

50

Layout Conventions - C#

Limit Line Length

Our eyes are more comfortable when reading tall and narrow columns of text



51

Layout Conventions

Limit Line Length

Should limit the line length to around **80 characters**

```
01 // bad
02 $my_email->set_from('test@email.com')->add_to('programming@gmail.com')->set_subject('Methods Chained')->
03
04 // good
05 $my_email
06     ->set_from('test@email.com')
07     ->add_to('programming@gmail.com')
08     ->set_subject('Methods Chained')
09     ->set_body('Some long message')
10     ->send();
11
12 // bad
13 $query = "SELECT id, username, first_name, last_name, status FROM users LEFT JOIN user_posts USING(users
14
15 // good
16 $query = "SELECT id, username, first_name, last_name, status
17 FROM users
18 LEFT JOIN user_posts USING(users.id, user_posts.user_id)
19 WHERE post_id = '123'";
```

Avoid displaying scrollbar

52

Layout Conventions Avoid Deep Nesting

```
function do_stuff() {
// ...
    if (is_writable($folder)) {
        if ($fp = fopen($file_path, 'w')) {
            if ($stuff = get_some_stuff()) {
                if (fwrite($fp, $stuff)) {
                    // ...
                } else {
                    return false;
                }
            } else {
                return false;
            }
        } else {
            return false;
        }
    } else {
        return false;
    }
}
```

```
function do_stuff() {
// ...
    if (!is_writable($folder)) {
        return false;
    }
    if (!$fp = fopen($file_path, 'w')) {
        return false;
    }
    if (!$stuff = get_some_stuff()) {
        return false;
    }
    if (fwrite($fp, $stuff)) {
        // ...
    } else {
        return false;
    }
}
```

53

Layout Conventions

Code grouping

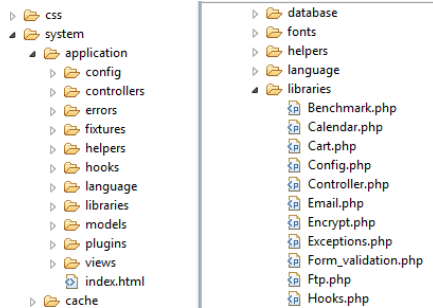
Keep tasks within **separate blocks** of code, with some spaces between them.

```
01 // get list of forums
02 $forums = array();
03 $r = mysql_query("SELECT id, name, description FROM forums");
04 while ($d = mysql_fetch_assoc($r)) {
05     $forums []= $d;
06 }
07
08 // load the templates
09 load_template('header');
10 load_template('forum_list', $forums);
11 load_template('footer');
```

54

Layout Conventions

File and Folder Organization



55

Commenting Conventions

- Place the comment on a separate line, not at the end of a line of code.
- Begin comment text with an uppercase letter.
- End comment text with a period.
- Insert one space between the comment delimiter (//) and the comment text.

```
C#
// The following declaration creates a query. It does not run
// the query.
```

56

Commenting Conventions

Avoid Obvious Comments

```
1 // get the country code
2 $country_code = get_country_code($_SERVER['REMOTE_ADDR']);
3
4 // if country code is US
5 if ($country_code == 'US') {
6
7     // display the form input for state
8     echo form_input_state();
9 }
```

If you must comment on that code, try to combine them instead

```
1 // display state selection for US users
2 $country_code = get_country_code($_SERVER['REMOTE_ADDR']);
3 if ($country_code == 'US') {
4     echo form_input_state();
5 }
```

57

Best practices

Good Programming Practices in C#

58

Use object initializer

- Use object initializer

```
//Avoid
Test test = new Test();
test.Id = 1;
test.Name = "faisal";
```

```
//Do
var test = new Test
{
    Id = 1,
    Name = "faisal"
};
```

59

initialize arrays

- Use the concise syntax when you initialize arrays on the declaration line.

```
// Preferred syntax. Note that you cannot use var here instead of string[].
string[] vowels1 = { "a", "e", "i", "o", "u" };

// If you use explicit instantiation, you can use var.
var vowels2 = new string[] { "a", "e", "i", "o", "u" };

// If you specify an array size, you must initialize the elements one at a time.
var vowels3 = new string[5];
vowels3[0] = "a";
vowels3[1] = "e";
// And so on.
```

60

Use StringBuilder to append strings

- To append strings in loops, especially when you are working with large amounts of text, use a [StringBuilder](#) object.

```
var phrase = "lalalalalalalalalalalalalalalalalalalalalalalalalalalal";  
var manyPhrases = new StringBuilder();  
for (var i = 0; i < 10000; i++)  
{  
    manyPhrases.Append(phrase);  
}  
//Console.WriteLine("tra" + manyPhrases);
```

61

Implicitly Typed Local Variables

- Use **implicit typing** for local variables when the type of the variable is obvious from the right side of the assignment, or when the precise type is not important.

```
// When the type of a variable is clear from the context, use var
// in the declaration.
var var1 = "This is clearly a string.";
var var2 = 27;
var var3 = Convert.ToInt32(Console.ReadLine());
```

62

Implicitly Typed Local Variables

- **DO NOT** use `var` when the type is not apparent from the right side of the assignment.

```
// When the type of a variable is not clear from the context, use an
// explicit type.
int var4 = ExampleClass.ResultSoFar();
```

```
int var4 = ExampleClass.ResultSoFar();
```

63

Implicitly Typed Local Variables

- Use implicit typing to determine the type of the loop variable in for and foreach loops.

```
var phrase = "lalalalalalalalalalalalalalalalalalalalalalalalal";  
var manyPhrases = new StringBuilder();  
for (var i = 0; i < 10000; i++)  
{  
    manyPhrases.Append(phrase);  
}  
  
//Console.WriteLine("tra" + manyPhrases);
```

64

Check Null or Empty

- To check null or empty condition.

```
//Avoid
var varName = "faisal";
if (varName != null && varName != "")
{
    //code
}

//Do
var varName = "faisal";
if (!string.IsNullOrEmpty(varName))
{
    //code
}
```

65

try-catch statement

- Use a [try-catch](#) statement for most exception handling.

```
static string GetValueFromArray(string[] array, int index)
{
    try
    {
        return array[index];
    }
    catch (System.IndexOutOfRangeException ex)
    {
        Console.WriteLine("Index is out of range: {0}", index);
        throw;
    }
}
```

66

Use **using** statement

- Simplify your code by using the C# [using statement](#).
 - If you have a [try-finally](#) statement in which the only code in the finally block is a call to the [Dispose](#) method, use a using statement instead.

67

Use **using** statement

```
// This try-finally statement only calls Dispose in the finally block.
Font font1 = new Font("Arial", 10.0f);
try
{
    byte charset = font1.GdiCharSet;
}
finally
{
    if (font1 != null)
    {
        ((IDisposable)font1).Dispose();
    }
}

// You can do the same thing with a using statement.
using (Font font2 = new Font("Arial", 10.0f))
{
    byte charset = font2.GdiCharSet;
}
```

68

Use **using** statement

```
string connectionString = "...";

using (SqlConnection con = new SqlConnection(connectionString)) {

    con.Open();
    using (SqlCommand command =
        new SqlCommand("SELECT * FROM Customers", con))
        using (SqlDataReader reader = command.ExecuteReader()) {

            while (reader.Read()) {
                Console.WriteLine("{0} {1} {2}",
                    reader.GetInt32(0),
                    reader.GetString(1));
            }
        }
}
```

69

&& and || operators

- To avoid exceptions and increase performance by skipping unnecessary comparisons, use [&&](#) instead of [&](#) and [||](#) instead of [|](#) when you perform comparisons

70

```
Console.WriteLine("Enter a dividend: ");
var dividend = Convert.ToInt32(Console.ReadLine());

Console.WriteLine("Enter a divisor: ");
var divisor = Convert.ToInt32(Console.ReadLine());

// If the divisor is 0, the second clause in the following condition
// causes a run-time error. The && operator short circuits when the
// first expression is false. That is, it does not evaluate the
// second expression. The & operator evaluates both, and causes
// a run-time error when divisor is 0.
if ((divisor != 0) && (dividend / divisor > 0))
{
    Console.WriteLine("Quotient: {0}", dividend / divisor);
}
else
{
    Console.WriteLine("Attempted division by 0 ends up here.");
}
```

71

DRY Principle **Avoid duplicated code!**

- DRY stands for **Don't Repeat Yourself**.
- Also known as DIE: Duplication is Evil.

Extract repeated elements

```
1 $this->load->view('includes/header');
2
3 $this->load->view($main_content);
4
5 $this->load->view('includes/footer');
```

72

Avoid writing long methods

- Avoid writing very long methods.
- A method should typically have **1~25 lines** of code.
- If a method has more than 25 lines of code, you must consider **re-factoring into separate methods**.

73

Method name should tell what it does

- Method name should tell what it does.
 - If the method name is obvious, there is no need of documentation explaining what the method does.

Good:

```
void SavePhoneNumber ( string phoneNumber )
{
    // Save the phone number.
}
```

Not Good:

```
// This method will save the phone number.
void SaveDetails ( string phoneNumber )
{
    // Save the phone number.
}
```

74

A method should do only one job

- A method should do only 'one job'. Do not combine more than one job in a single method, even if those jobs are very small.

Not Good:

```
// Save address and send an email to the supervisor to inform that
// the address is updated.
SaveAddress ( address, email );

void SaveAddress ( string address, string email )
{
    // Job 1.
    // Save the address.
    // ...

    // Job 2.
    // Send an email to inform the supervisor that the address is changed.
    // ...
}
```

75

A method should do only one job

Good:

```
// Save the address.
SaveAddress ( address );

// Send an email to the supervisor to inform that the address is updated.
SendEmail ( address, email );

void SaveAddress ( string address )
{
    // Save the address.
    // ...
}

void SendEmail ( string address, string email )
{
    // Send an email to inform the supervisor that the address is
    // changed.
    // ...
}
```

76

Avoid Too Many Parameters

```
//avoid
public void Checkout(string shippingName, string
shippingCity, string shippingState, string shippingZip,
string billingName, string billingCity, string
billingState, string billingZip) { }
```

```
//DO
public void Checkout(
    ShippingAddress shippingAddress,
    BillingAddress billingAddress) { }
```

Declare a class instead of too many parameters

77

Magic Number

- A 'magic number' is a literal value that appears in a program. For example:

```
total = 1.08 * price;
```

- 1.08 is a magic number (what it means?)
- Should replace magic numbers with **NamedConstants**, e.g.

```
const double TaxRate = 1.08;
total = TaxRate * price;
```

REF: <http://wiki.c2.com/?MagicNumber>

78

Magic Number

- By replace with Named constants, it has a couple of advantages:
 - The fuller name makes it **clear**.
 - If you **change the value** you only need to change the value once in the constant declaration.

79

Always watch for unexpected values

- Always watch for unexpected values.
 - For example, if you are using a parameter with 2 possible values, never assume that if one is not matching then the only possibility is the other value.

80

Not good

```
If ( memberType == eMemberTypes.Registered )
{
    // Registered user... do something...
}
else
{
    // Guest user... do something...

    // If we introduce another user type in
    future, this code will fail and will not be
    noticed.
}
```

81

Good

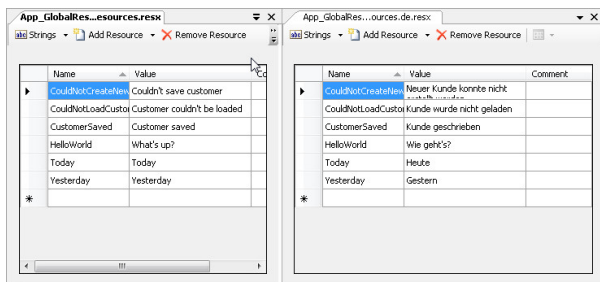
```
If ( memberType == eMemberTypes.Registered )
{
    // Registered user... do something...
}
else if ( memberType == eMemberTypes.Guest )
{
    // Guest user... do something...
}
else
{
    // Un expected user type. Throw an exception
    throw new Exception ("Un expected value " +
        memberType.ToString() + "'.")

    // If we introduce a new user type in future, we can
    easily find the problem here.
}
```

82

Do not hardcode strings

Do not hardcode strings. Use **resource files**.



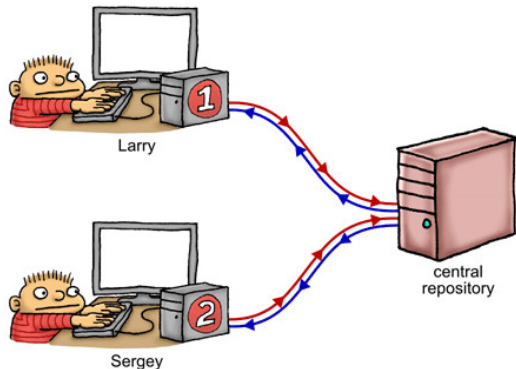
83

Nội dung

1. Quy trình viết mã nguồn
2. Tái cấu trúc mã nguồn
3. Coding conventions
4. Quản lý mã nguồn

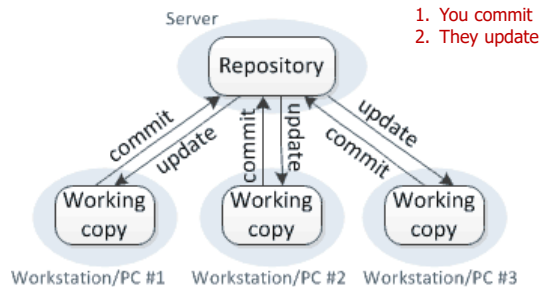
84

Làm thế nào để cùng xây dựng mã nguồn / dự án?



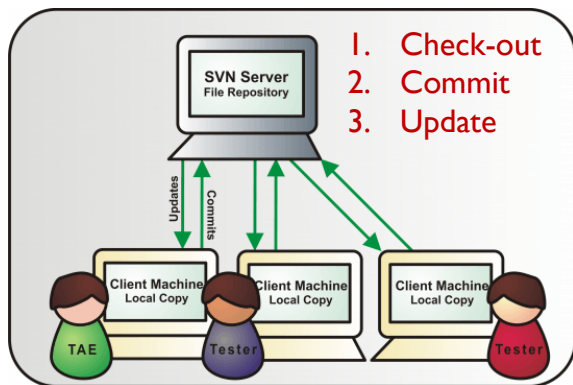
85

Quản lý phiên bản tập trung Centralized version control



86

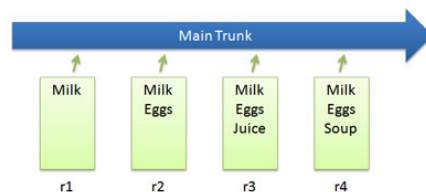
Important activities



87

Check in / Commit

Basic Checkins



Check in: Upload a file to the repository (if it has changed).
The file gets a new revision number.

88

Các công cụ cho hệ thống quản lý phiên bản tập trung

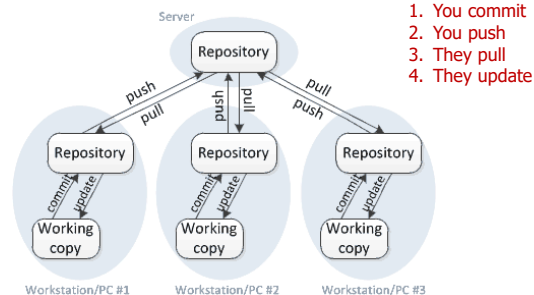
- **Client:**
 - Tortoise SVN
- **Server (Subversion Hosting):**
 - Tự cài đặt server
 - RiouxSVN (<https://riouxsvn.com>)
 - ...



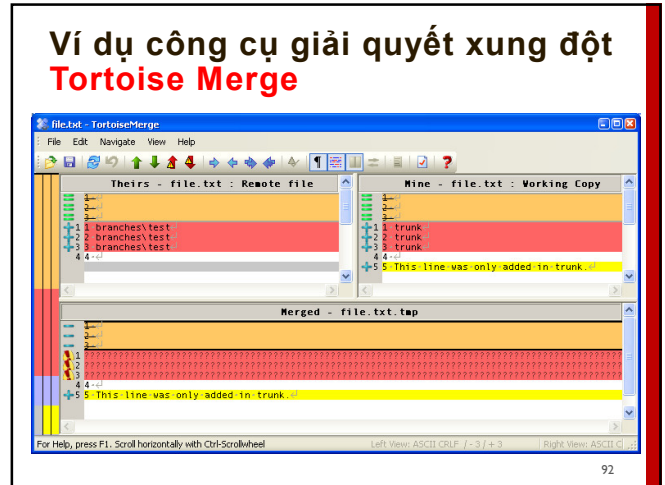
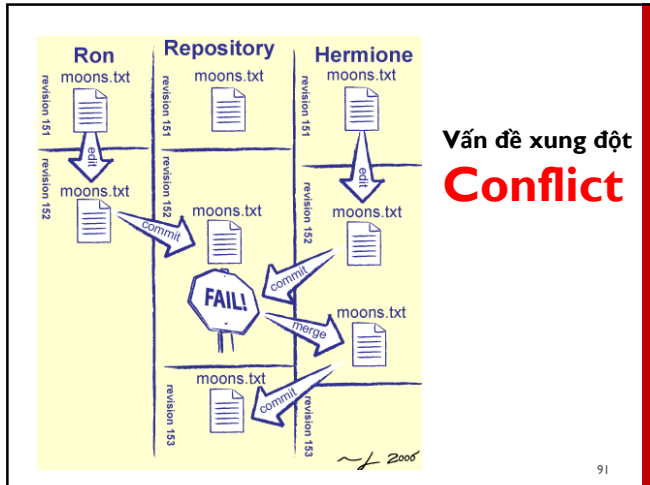
89

Quản lý phiên bản phân tán

Distributed version control



90



Tham khảo

[1] Ian Sommerville, *Software Engineering*, 9th ed., Addison-Wasley, 2011.

[2] Pankaj Jalote, *A Concise Introduction to Software Engineering*, Springer, 2008.

Websites:

- <https://sourcemaking.com/refactoring/smells>
- https://backlog.com/git-tutorial/vn/intro/intro1_1.html
- Các tài liệu tham khảo ở phần Coding Conventions